

```

import numpy as np

class Madaline:

    def __init__(self, n_inputs, n_hidden, n_outputs, learning_rate=0.01, epochs=100):
        self.n_inputs = n_inputs
        self.n_hidden = n_hidden
        self.n_outputs = n_outputs
        self.learning_rate = learning_rate
        self.epochs = epochs

        # Initialize weights and biases for the hidden layer
        # Small random values for weights to break symmetry and aid learning
        self.weights_hidden = np.random.randn(n_inputs, n_hidden) * 0.01
        self.bias_hidden = np.zeros((1, n_hidden))

        # Initialize weights and biases for the output layer
        self.weights_output = np.random.randn(n_hidden, n_outputs) * 0.01
        self.bias_output = np.zeros((1, n_outputs))

        print(f"Madaline model initialized with: ")
        print(f" Input neurons: {self.n_inputs}")
        print(f" Hidden neurons: {self.n_hidden}")
        print(f" Output neurons: {self.n_outputs}")
        print(f" Learning Rate: {self.learning_rate}")
        print(f" Epochs: {self.epochs}")
        print(f" Hidden layer weights shape: {self.weights_hidden.shape}")
        print(f" Hidden layer bias shape: {self.bias_hidden.shape}")
        print(f" Output layer weights shape: {self.weights_output.shape}")
        print(f" Output layer bias shape: {self.bias_output.shape}")

    def _feedforward(self, X):
        # Calculate linear output for the hidden layer
        hidden_linear_output = np.dot(X, self.weights_hidden) + self.bias_hidden

        # Apply step function for hidden layer activations
        # Output 1 if linear output >= 0, else -1
        hidden_activations = np.where(hidden_linear_output >= 0, 1, -1)

        # Calculate linear output for the output layer
        output_linear_output = np.dot(hidden_activations, self.weights_output) + self.bias_output

        return hidden_activations, output_linear_output

    def fit(self, X, y):
        n_samples = X.shape[0]
        self.losses = []

        for epoch in range(self.epochs):
            # Perform a feedforward pass
            hidden_activations, output_linear_output = self._feedforward(X)

            # Calculate the output layer error
            # Ensure y has the correct shape for broadcasting with output_linear_output
            errors_output = y.reshape(-1, self.n_outputs) - output_linear_output

            # Calculate and store MSE loss
            loss = (errors_output ** 2).mean() / 2 # MSE divided by 2 for gradient calculation convenience
            self.losses.append(loss)
            print(f"Epoch {epoch + 1}/{self.epochs}, Loss: {loss:.4f}")

            # Update the weights and biases of the output layer
            # Delta rule for output layer
            self.weights_output += self.learning_rate * np.dot(hidden_activations.T, errors_output) / n_samples
            self.bias_output += self.learning_rate * np.sum(errors_output, axis=0, keepdims=True) / n_samples

            # Calculate the error signal for the hidden layer (backpropagation)
            # This is the error from the output layer weighted by the output layer's weights
            delta_hidden = np.dot(errors_output, self.weights_output.T)

            # Update the weights and biases of the hidden layer
            # The adaptation for the hidden layer's delta rule is trickier for step functions.
            # For MADALINE, usually, the weights of hidden neurons that produce wrong outputs are adjusted
            # (e.g., based on the 'minimum disturbance principle').
            # Here, we'll use a simplified delta rule for demonstration, assuming the hidden units are 'Adaline-like'
            # and we are trying to reduce the error on their linear outputs, which is implied by delta_hidden.
            # Note: A true Madaline learning rule for hidden layers is more complex and often involves a specific
            # mechanism for correcting misclassified patterns by adjusting a minimal number of hidden weights.

            # Update hidden weights and bias based on the propagated error
            self.weights_hidden += self.learning_rate * np.dot(X.T, delta_hidden) / n_samples

```

```

self.weights_hidden += self.learning_rate * np.dot(X.T, delta_hidden) / n_samples
self.bias_hidden += self.learning_rate * np.sum(delta_hidden, axis=0, keepdims=True) / n_samples

def predict(self, X):
    # Perform a feedforward pass
    hidden_activations, output_linear_output = self._feedforward(X)

    # Apply step function to the output linear output to get class predictions
    # Output 1 if linear output >= 0, else -1
    predictions = np.where(output_linear_output >= 0, 1, -1)

    return predictions

```

```

import numpy as np

# 1. Define the XOR problem dataset
X_madaline = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y_madaline = np.array([-1, 1, 1, -1]) # XOR labels: -1 for (0,0), (1,1); 1 for (0,1), (1,0)

# Reshape y_madaline for the fit method (single output neuron)
y_madaline = y_madaline.reshape(-1, 1)

print("X_madaline shape:", X_madaline.shape)
print("y_madaline shape:", y_madaline.shape)

# 2. Instantiate the Madaline class
# For XOR: n_inputs=2, n_hidden=2 (or more), n_outputs=1
model_madaline = Madaline(n_inputs=2, n_hidden=2, n_outputs=1, learning_rate=0.01, epochs=2000)

# 3. Train the Madaline model
print("\nTraining Madaline model...")
model_madaline.fit(X_madaline, y_madaline)
print("Madaline model training complete.")

# 4. Make predictions on the X_madaline dataset
predictions_madaline = model_madaline.predict(X_madaline)

print("\n--- Madaline Model Results ---")
print("Input (X_madaline):\n", X_madaline)
print("Actual Labels (y_madaline):\n", y_madaline.flatten())
print("Predicted Labels (predictions_madaline):\n", predictions_madaline.flatten())

# Evaluate accuracy (for XOR, ideally 100% after training)
accuracy = np.mean(predictions_madaline == y_madaline) * 100
print(f"\nAccuracy: {accuracy:.2f}%")

```

```
Epoch 1990/2000, Loss: 0.5000
Epoch 1991/2000, Loss: 0.5000
Epoch 1992/2000, Loss: 0.5000
Epoch 1993/2000, Loss: 0.5000
Epoch 1994/2000, Loss: 0.5000
Epoch 1995/2000, Loss: 0.5000
Epoch 1996/2000, Loss: 0.5000
Epoch 1997/2000, Loss: 0.5000
Epoch 1998/2000, Loss: 0.5000
Epoch 1999/2000, Loss: 0.5000
Epoch 2000/2000, Loss: 0.5000
Madaline model training complete.
```

```
--- Madaline Model Results ---
```

```
Input (X_madaline):
```

```
[[0 0]
 [0 1]
 [1 0]
 [1 1]]
```

```
Actual Labels (y_madaline):
```

```
[-1  1  1 -1]
```

```
Predicted Labels (predictions_madaline):
```

```
[-1 -1 -1 -1]
```

```
Accuracy: 50.00%
```

```
print("\n--- Training Details ---")
```

```
print("Training Losses (last 20 epochs):", model_madaline.losses[-20:]) # Print last 20 losses for brevity
```

```
print("\nFinal Hidden Layer Weights:\n", model_madaline.weights_hidden)
```

```
print("Final Hidden Layer Bias:\n", model_madaline.bias_hidden)
```

```
print("\nFinal Output Layer Weights:\n", model_madaline.weights_output)
```

```
print("Final Output Layer Bias:\n", model_madaline.bias_output)
```

```
--- Training Details ---
```

```
Training Losses (last 20 epochs): [np.float64(0.5), np.float64(0.5), np.float64(0.5), np.float64(0.5), np.float64(0.5), np.f
```

```
Final Hidden Layer Weights:
```

```
[[ -3.78251487e-05 -5.11341529e-03]
 [-5.45441696e-05 -3.23052668e-02]]
```

```
Final Hidden Layer Bias:
```

```
[[ -8.52431074e-05 -1.88052510e-04]]
```

```
Final Output Layer Weights:
```

```
[[ -0.07960356]
 [ 0.07896613]]
```

```
Final Output Layer Bias:
```

```
[[ -0.00063743]]
```

```
print("\nFull Training Losses:\n", model_madaline.losses)
```

```
Full Training Losses:
```

```
[np.float64(0.5011678084740427), np.float64(0.5000036752312013), np.float64(0.4985878100991043), np.float64(0.4936408976266
```